

Lab 04: GPG Key Generation, Encryption, and Signing Report

Exercise Objective

In this exercise, I learned how to use GPG (GNU Privacy Guard) for basic file security. First, I checked whether GPG was installed on my system. Then, I created a key pair and practiced exporting and importing a public key. I also encrypted a file, signed it digitally, and later decrypted the file and verified the signature. This practical exercise helped me understand the basic use of GPG for keeping files secure and checking their authenticity.

```
(kali㉿kali)-[~/CS-Vaje/lab03]
└─$ sudo apt install gnupg
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gnupg is already the newest version.
Upgrading:
  dirnmgr  gnupg-l10n  gpg-agent  gpg-wks-client  gpgsm  gnupg-utils  gnupg  gnupg-wks-client  gpgconf  gpg-agent  gpgconf
  gnupg  gnupg-utils  gpg-agent  gpgconf  gpg-agent  gpgconf
  gnupg  gnupg-utils  gpg-agent  gpgconf  gpg-agent  gpgconf
Summary: 11 upgradeable packages
Upgrading: 9, Installing: 0, Removing: 0, Not Upgrading: 827
Download size: 3,149 kB
Space needed: 9,216 B / 50.9 GB available
Continue? [Y/n] y
Get:1 http://mirror.karneval.cz/pub/linux/kali kali-rolling/main amd64 gpg-w
ks-client amd64 2.4.9-7 [106 kB]
Get:3 http://mirror1.sox.rs/kali/kali kali-rolling/main amd64 gnupg-utils am
d64 2.4.9-7 [192 kB]
Get:8 http://kali.mirror.garr.it/kali kali-rolling/main amd64 gpgsm amd64 2.
4.9-7 [274 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 gpg-agent amd64 2.4.
9-7 [270 kB]
Get:2 http://mirror.pyratelan.org/kali kali-rolling/main amd64 dirnmgr amd64
2.4.9-7 [382 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 gpg amd64 2.4.9-7 [6
36 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 gpgconf amd64 2.4.9-
7 [126 kB]
Get:7 http://kali.download/kali kali-rolling/main amd64 gnupg all 2.4.9-7 [4
15 kB]
Get:9 http://kali.download/kali kali-rolling/main amd64 gnupg-l10n all 2.4.9
-7 [748 kB]
```

1. Generate GPG Key Pair

```

(kali㉿kali)-[~/CS-Vaje/lab03] : 100000, 'cracked': None)
$ gpg --full-generate-key: 100000, 'cracked': None)
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
/home/kali - ~/Desktop/Crypto
gpg: keybox '/home/kali/.gnupg/pubring.kbx' created
Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal (~/Desktop/Crypto)
  (3) DSA (sign only).py
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*!o
 (10) ECC (sign only)
 (14) Existing key from card se (.csv) 'wordlist.txt'
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Wed 18 Aug 2027 04:03:30 PM EDT
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: vk
Email address: vk@gmail.com
Comment: vk-is -vk
You selected this USER-ID:
    "vk (vk-is -vk) <vk@gmail.com>"

```

2. Generate Dona GPG Key Pair

```

(kali㉿kali)-[~/CS-Vaje/Lab03] : 100000, Cracked: None]
$ gpg --full-generate-key : 100000, Cracked: None]
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
~/.gnupg
~/Desktop/Crypto
Please select what kind of key you want:
  (1) RSA and RSA (kali)
  (2) DSA and Elgamal
  (3) DSA (sign only) ~/Desktop/Crypto
  (4) RSA (sign only) py
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only) ~/Desktop/Crypto
 (14) Existing key from card
Your selection? 1: word_database.csv wordlist.txt
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096/py
Requested keysize is 4096 bits
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0) 1y
Key expires at Wed 18 Aug 2027 04:07:19 PM EDT
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: dona
Email address: dona@example.com
Comment: Dona
You selected this USER-ID:
    "dona (Dona) <dona@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number

```

3. Export and Import Public Key

```

(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --armor --export vk@ert.com > vk_pubkey.asc
[username: 'mallory', 'iterations': 100000, 'cracked': None]
(kali㉿kali)-[~/CS-Vaje/lab03] 100000, 'cracked': None)
$ gpg --armor --export dona@example.com > dona_pubkey.asc
[username: 'deter', 'iterations': 100000, 'cracked': None]
(kali㉿kali)-[~/CS-Vaje/lab03] 'cracked': None)
$ gpg import vk_pubkey.asc 0.23 ms
gpg: WARNING: no command supplied. Trying to guess what you mean ...
usage: gpg [options] [filename]
       gpg [options] --keyring /path/to/keyring
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --import vk_pubkey.asc
gpg: key 074137CAF84D06A1: "vk (vk-vk) <vk@ert.com>" not changed
gpg: Total number processed: 1
gpg: unchanged: 1
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --import dona_pubkey.asc
gpg: key 56FC8BB9678CF841: "dona (Dona) <dona@example.com>" not changed
gpg: Total number processed: 1
gpg: unchanged: 1
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 2 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 2u
gpg: next trustdb check due at 2027-08-18
/home/kali/.gnupg/pubring.kbx

pub   rsa4096 2026-08-18 [SC] [expires: 2027-08-18]
      6B8E1AFC71CED4ACB31E8AFC074137CAF84D06A1
uid           [ultimate] vk (vk-vk) <vk@ert.com>
sub   rsa4096 2026-08-18 [E] [expires: 2027-08-18]

pub   rsa4096 2026-08-18 [SC] [expires: 2027-08-18]
      6004EA0A69323B20D8096A0B56FC8BB9678CF841
uid           [ultimate] dona (Dona) <dona@example.com>
sub   rsa4096 2026-08-18 [E] [expires: 2027-08-18]

```

5. Preparing the Message

```

(kali㉿kali)-[~/CS-Vaje/lab03]
$ echo " To:dona@examplr.com
dquote> from:vk@ert.com ~/Desktop/Crypto
dquote> Date:\$(date).py
dquote> Secret message: Confidential message from Vk to Dona" >message.txt
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --encrypt --sign --armor --recipient dona@example.com message.txt

```

6. Decrypting and Verifying the Signature

```

(kali@kali)-[~/CS-Vaje/lab03]
└─$ gpg --decrypt message.txt.asc > decrypted_message.txt
gpg: encrypted with rsa4096 key, ID C97608F1EFBCA438, created 2026-08-18
      "dona (Dona) <dona@example.com>"
gpg: Signature made Tue 18 Aug 2026 04:27:33 PM EDT
gpg: Good signature from "vk (vk-vk) <vk@ert.com>" [ultimate]

(kali@kali)-[~/CS-Vaje/lab03]
└─$ cat decrypted_message.txt
To:dona@examplr.com
from:vk@ert.com
Date:$(date)
Secret message: Confidential message from Vk to Dona.

(kali@kali)-[~/CS-Vaje/lab03]
└─$

```

Additional Task

7. Revocation and Verifying Certificate

```

(kali@kali)-[~/CS-Vaje/lab03]
└─$ gpg --gen-revoke vk@ert.com > revoke.asc 'cracked': 'password123'
{username: 'peter', iterations: 100000, 'cracked': None}
sec: rsa4096/074137CAF84D06A1 2026-08-18 vk (vk-vk) <vk@ert.com>
PEKID: full wordlist attack: 0.23 ms
Create a revocation certificate for this key? (y/N) y
Please select the reason for the revocation:
  0 = No reason specified
  1 = Key has been compromised
  2 = Key is superseded
  3 = Key is no longer used
  Q = Cancel
Your decision? 3
Enter an optional description; end it with an empty line:
>
Reason for revocation: Key is no longer used
(No description given)
Is this okay? (y/N) y
ASCII armored output forced.
Revocation certificate created.

Please move it to a medium which you can hide away; if Mallory gets
access to this certificate he can use it to make your key unusable.
It is smart to print this certificate and store it away, just in case
your media become unreadable. But have some caution: The print system of
your machine might store the data and make it available to others!

```

```

analysar.py
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --clearsign message.txt
file 'message.txt.asc' exists. Overwrite? (y/N) y
analysar.py password_database.csv wordlist.txt
(kali㉿kali)-[~/CS-Vaje/lab03]
$ gpg --verify message.txt.asc
gpg: Signature made Tue 18 Aug 2026 04:36:44 PM EDT
gpg:      using RSA key 6B8E1AFC71CED4ACB31E8AFC074137CAF84D06A1
gpg: Good signature from "vk (vk-vk) <vk@ert.com>" [ultimate]
gpg: WARNING: not a detached signature; file 'message.txt' was NOT verified!

```

Questions

Difference between encryption and signing?

Encryption and signing serve different security goals. Encryption provides confidentiality: only the intended recipient can read the message. For example, when VK encrypts a message for Dona, he uses Dona's public key; only Dona can decrypt it with her private key, so the message is hidden from everyone else.

Digital signing provides authentication and integrity: it proves who sent the message and that it was not altered in transit. When VK signs a message, he uses his private key to create the signature. Anyone with VK's public key can verify the signature, confirming the message came from VK and has not been modified.

Role of public and private keys?

GPG uses asymmetric cryptography: each user has a public key and a private key with complementary roles.

Public key (shared): You distribute this key widely or publish it on a keyserver. Others use your public key to encrypt messages intended for you (only your private key can decrypt them). They also use your public key to verify signatures you create.

Private key (secret): You must keep this key secure and protected by a strong passphrase. You use your private key to decrypt messages encrypted with your public key and to create digital signatures that prove a message came from you.

In this exercise: VK's public key was given to Dona so she could verify his signatures; VK's private key was used to sign messages. Dona's public key was given to VK so he could encrypt messages to her; Dona's private key was used to decrypt those messages.

What happens when an encrypted file is modified?

If an encrypted file is modified after encryption, decryption will fail. GPG includes integrity protection (a checksum or MAC) inside the encrypted data that depends on the exact ciphertext. Even a single-byte change breaks the checksum, so GPG will report an error such as "gpg: decryption failed: Bad session key" and refuse to decrypt.

This behaviour defends against tampering and man-in-the-middle attacks. If the integrity check were bypassed and decryption forced, the result would be corrupted, unreadable output. If the file was also signed, signature verification would fail ("Bad signature") because the signed contents no longer

match the signature. Together, encryption integrity and signatures provide confidentiality and integrity.